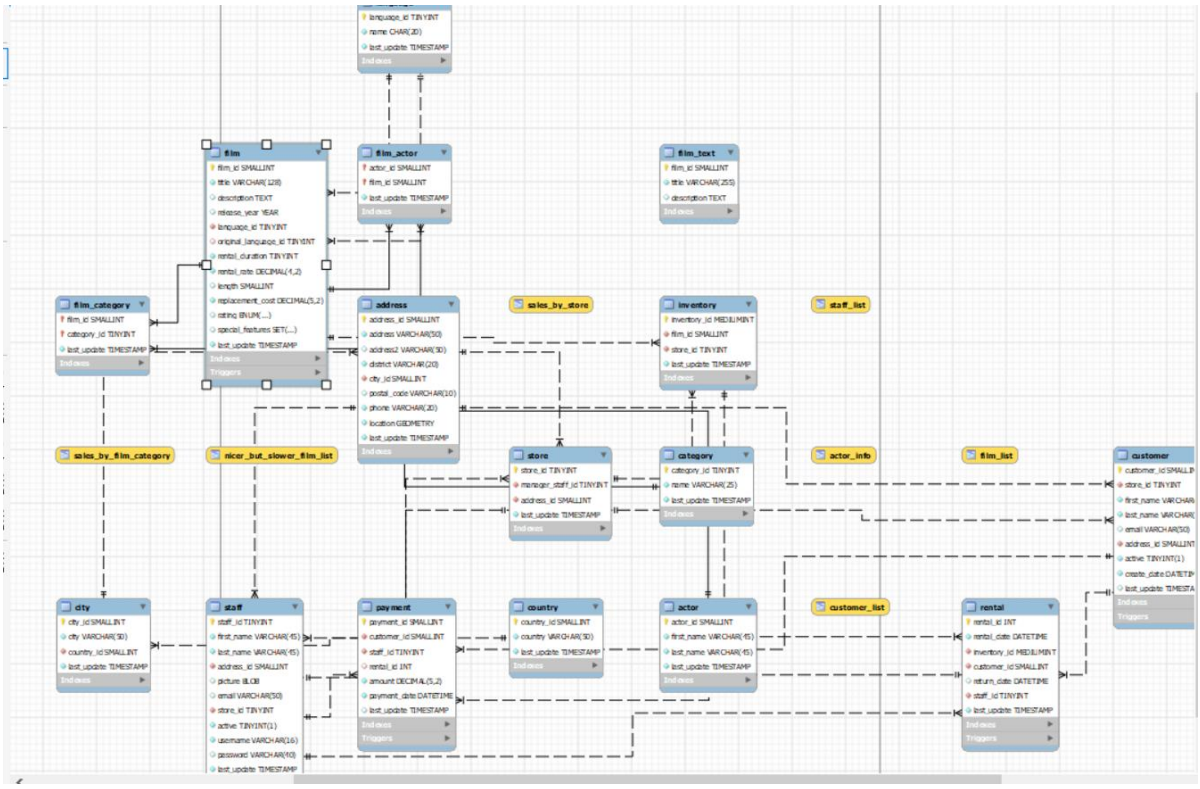
Exploring movies according to personal information

Yinan Gu (V00990694)

Introduction

In this project, I will focus on each user's renting history and help them find other moves of their preference from people in their countries.
The database used in the project is Sakila Sample Database. It contains 16 tables:

The overall view of the relationships between them is as follows:



Analysis

My idea to explore movies is to extract all the films' titles and film_ids, inventory_id and store_id of the inventories that were rented by customers in the uses's country and the films are of the user's favorite category and have the user's favorite actor.
This idea can be decomposed into 2 parts:

- Find the user's favorite category and actor
- Find the films and related inventories with the user's favorite actor and category and the related inventories are rented by customers in the user's country

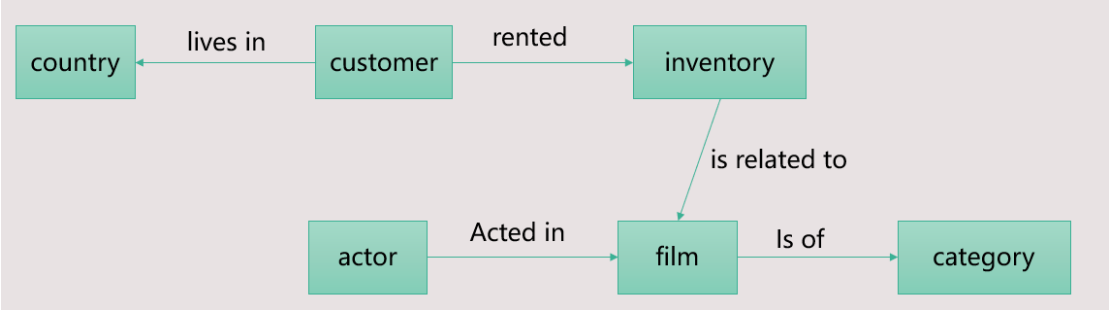
Data modeling

For each requirement, I will need different information. In the step to find the user's country, favorite category and actor, I will need country, customer, film, actor and category; for the exploration stage, I will need all previous information plus inventory. And for each table, I need country_id and country name for table country; customer_id, first_name and last_name for customer; inventory_id, store_id for table inventory; film_id and film title for table film; actor_id, first_name, last_name for table actor; category_id and category name for table category.
Also, the relationship between them are:

- a customer lives in one country

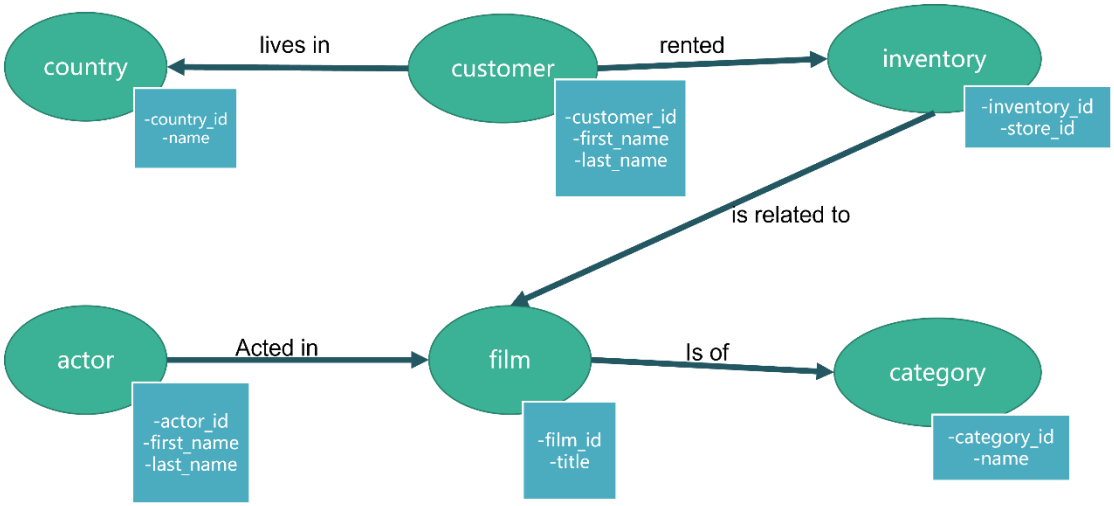
- the customer rented inventories
- an inventory is related to one film
- a film is of a category and have many actors

As I have listed all 6 tables and their related attributes, I built the conceptual data model according to the relationships between them using the bottom-up method as follows:



Logical model:

After embedding the attributes, I generated the logical graph model as follows:



Minimality & completeness

- Minimality
  - There are no duplicates in attributes or entities
  - All attributes are used for querying;
  - IDs like country_id, customer_id, actor_id, film_id are used to differentiate the probable same names or titles.
- Completeness
  - For finding the user's country, favorite actor and category:
    - Find country node with given customer_id in customer
    - Get rental record through path: customer->inventory->film and path: actor->film->category to find most frequent actor and category
  - For finding the films and inventories:
    - Find the customer according to the country node
    - According to the properties in the actor and category to find information of all the films and related inventories

Queryability

- how natural the queries are to write
  - For finding the user's country, favorite actor and film category, we just need to locate the node customer with specific customer, then we can traverse the paths to find the user's country and his film records;
  - For getting the film and inventories based on the user's information, we have two paths, the first one we have the "customer rented some inventories that are related to some films of some category"; the second one is "some actor acted in the film".
- how quickly can we serve up some information that is relevant to our requirements
  - First, locating the customers according to the country node;
  - Then, with nodes actor and category together, we can locate the film nodes quickly;
- how well the schema makes use of the underlying model
  - For finding the user's information, it traversed all the nodes naturally;
  - For exploring the desired film information, it utilized the nodes country, actor and category to locate efficiently

Implementation

Preparing the tables (code in MySQL workbench):

%generate tables: customers, inventories, rented

```
use sakila;
CREATE TABLE customers SELECT customer_id,first_name,last_name FROM customer;
alter table customers add primary key (customer_id);
create table inventories select inventory_id,store_id from inventory;
alter table inventories add primary key (inventory_id);
create table rented select inventory_id,customer_id from rental;
alter table rented add foreign key (customer_id) references customers (customer_id);
alter table rented add foreign key (inventory_id) references inventories (inventory_id);
```

%generate tables: countries

```
create table c select customer_id,address_id from customer;
create table ad select address_id, city_id from address;
create table ci select city_id, country_id from city;
create table countries select country_id, country from country;
alter table countries add primary key(country_id) ;
```

%generate table: lives_in

```
create table try1 select c.customer_id,ad.city_id from c cross join ad on c.address_id=ad.address_id;
create table try2 select ci.country_id,try1.customer_id from try1 cross join ci on try1.city_id=ci.city_id;
alter table try2 rename lives_in ;
alter table lives_in add foreign key (customer_id) references customers (customer_id);
alter table lives_in add foreign key (country_id) references countries (country_id);
```

%generate tables: is_related_to, films

```
create table is_related_to select inventory_id,film_id from inventory;
```

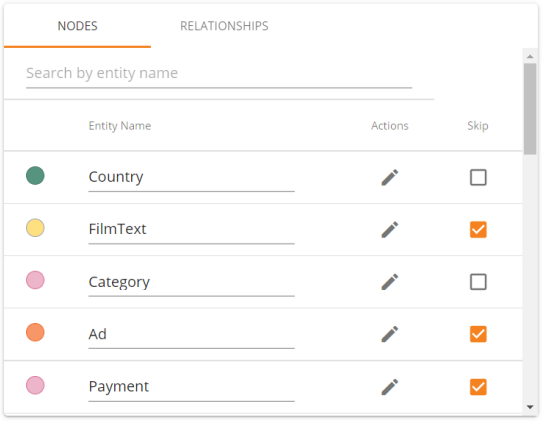
create table films select film_id, title from film;
alter table films add primary key(film_id);
alter table is_related_to add foreign key(inventory_id) references inventories(inventory_id);
alter table is_related_to add foreign key(film_id) references films(film_id);

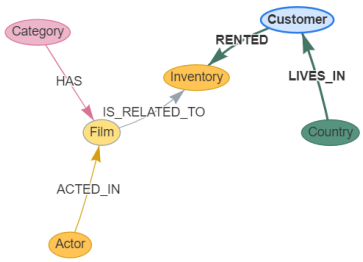
%create tables: actors, acted_in
create table actors select actor_id,first_name,last_name from actor;
alter table actors add primary key(actor_id);
create table acted_in select actor_id,film_id from film_actor;
alter table acted_in add foreign key(actor_id) references actors(actor_id);
alter table acted_in add foreign key(film_id) references films(film_id);

%create tables: categories, has
create table categories select category_id,name from category;
alter table categories add primary key(category_id);
create table has select film_id,category_id from film_category;
alter table has add foreign key(film_id) references films(film_id);
alter table has add foreign key(category_id) references categories(category_id);

processing in neo4j

After mapping from MySQL to Neo4j:





BACK TO START SAVE MAPPING NEXT

Get the countryId of the customer with customerId=1:

```
1 MATCH (coun)-[lives_in]→(c1:Customer{customerId:1})
2 return coun.countryId
```

coun.countryId
50

Get the category_id the user prefers: 4

```
MATCH(c1:Customer {customerId:1})-[rented]→(inv)←[is_related_to]-(fil)←[has]-(cate)
where cate.categoryId is not null
return cate.categoryId,count(*) order by count(*) desc
```

	cate.categoryId	count(*)
1	4	6
2	5	5
3	7	4
4	2	2
5	14	2
6	13	2
7		

Started streaming 14 records after 1 ms and completed after 4 ms.

Get the actor_id the user prefers: 37

```
MATCH(c1:Customer {customerId:1})-[rented]→(inv)←[is_related_to]-(fil)←[acted_in]-(ac)
where ac.actorId is not null
return ac.actorId,count(*) order by count(*) desc
```

	ac.actorId	count(*)
1	37	6
2	124	4
3	84	3
4	44	3
5	74	3
6	20	3
7		

Started streaming 108 records after 15 ms and completed after 19 ms.

Explore other films and related information:

```
match(cou:Country{countryId:50})-[lives_in]→(c)-[rented]→(inve)←[is_related_to]-(fi)←[acted_in]-(ac:Actor{actorId:37})
match(fi)←[has]-(categ:Category{categoryId:4})
where fi.title is not null
return fi.title,fi.filmId,inve.storeId
```

	fi.title	fi.filmId	inve.storeId
1	"PATIENT SISTER"	663	2
2	"PATIENT SISTER"	663	2
3	"PATIENT SISTER"	663	1
4	"PATIENT SISTER"	663	1
5	"PREJUDICE OLEANDER"	694	1

Started streaming 5 records after 1 ms and completed after 7 ms.

Conclusion

Generally, the queries wrote in cypher are not trivial and are very clear. However, during the mapping step of importing the tables from MySQL to Neo4j, the table names are modified and the relationships' direction also changed. In this way, I checked the properties first to write any queries.

From the results, we can see that by embedding the conditions extracted from the customer_id, we can collect all the film nodes and inventory nodes quickly.